Mini Project2

สำหรับส่งงาน Mini Project2 - สมาชิกกลุ่ม 2-3 คน

พัฒนาเว็บแอปพลิเคชัน ด้วย Modern Decoupled SPA (React 19 + Vite + TypeScript + React Router v7 + Zustand + TanStack Query) อย่างสมบูรณ์ เพื่อสะท้อนการแบ่งหน้าที่ระหว่าง Client State และ Server State

ข้อกำหนดในการพัฒนา

- Build Tool: Vite + React (TypeScript)
- Client-Side Routing: React Router v7 (ใช้งาน createBrowserRouter, <Outlet/>, useParams, <NavLink/>)
- State Management (Client): Zustand (พร้อมใช้งาน persist middleware สำหรับ LocalStorage)
- Server State & Caching: TanStack Query (React Query v5)
- Styling Framework: Tailwind CSS ร่วมกับ DaisyUI (หรือ UI Library อื่น เช่น shadcn/ui, Mantine, NextUI)
- Data Sources (อย่างน้อย 20 รายการ):
- Public REST API (เช่น Gutendex, TVMaze, FakeStoreAPI, TheMealDB, RAWG, Open Library) หรือ Mock API Server / JSON Server ที่จัดทำขึ้นเอง

ฟังก์ชันการทำงานหลักดังต่อไปนี้

1. หน้าหลักสำหรับนำเสนอข้อมูล
    - หน้าแสดง "รายการข้อมูล" (Items) อย่างน้อย 20 รายการ
    - แต่ละ "รายการ" สามารถเลือกดูรายละเอียดได้
2. หน้ารายละเอียดข้อมูล (Detail Page)
    - ตัวอย่าง: แสดงรายละเอียดสินค้า, ส่วนผสมและวิธีทำอาหาร, เรื่องย่อของหนังสือ

- ต้องมี Dynamic Route (เช่น `/product/:id`, `/recipe/:id`) เพื่อแสดงข้อมูลเชิงลึกของ "รายการ" ที่ถูกเลือก

3. ระบบคอลเลกชันส่วนตัว (Personal Collection System)**
    - ผู้ใช้ต้องสามารถ "บันทึก" หรือ "เก็บ" รายการที่สนใจไว้ในคอลเลกชันส่วนตัวได้
    - ตัวอย่าง: Wishlist, Favorites, My Cart, สูตรอาหารของฉัน
    - ข้อมูลคอลเลกชันนี้ ต้องถูกบันทึกไว้ เพื่อให้คงอยู่ในการโหลดครั้งต่อไป
4. หน้า About (ข้อมูลเกี่ยวกับผู้พัฒนา)
    - แสดงรูปถ่าย, ชื่อ-นามสกุล, ชื่อเล่น, รหัสนักศึกษา และหน้าที่ความรับผิดชอบในโปรเจกต์ของสมาชิกทุกคน
    - ระบุที่มาของข้อมูล (API Attribution) พร้อมแนบลิงก์ไปยังเอกสารของ API ที่นำมาใช้
5. ผู้พัฒนานำเสนอ และ พัฒนา อีก 1 หน้า หน้าพิเศษที่พัฒนาเพิ่มขึ้นมาให้สอดคล้องกับระบบ (Custom Extended Page) ให้แต่ละกลุ่มเลือกพัฒนาเพิ่มอีก 1 หน้า เพื่อเสริมความสมบูรณ์ของระบบ

ข้อกำหนดเชิงคุณภาพ (Non-Functional Requirements)

Architecture Integrity:

- **ห้าม** นำข้อมูลทั้งหมดจากเซิร์ฟเวอร์ไปเก็บซ้ำซ้อนใน Zustand
- Zustand ใช้สำหรับ Client State และ ID ของคอลเลกชันเท่านั้น
- TanStack Query ดูแลเรื่อง Network Data, Caching (ตั้งค่า staleTime เหมาะสม)

State Feedback (UI/UX):

- มีการจัดการสถานะ isLoading (แสดง Skeleton Loader หรือ Loading Spinner) ชัดเจนทุกจุดที่มีการต่อ API
- มีการจัดการสถานะ isError หรือ Network Failure พร้อมปุ่มกดลองใหม่อีกครั้ง (Retry Button)

- ออกแบบแบบ **Responsive Design** แสดงผลสมบูรณ์ทั้งบนมือถือ แท็บเล็ต และเดสก์ท็อป

TypeScript Strictness:

- กำหนด Type/Interface ให้กับข้อมูล API ทุกตัวอย่างละเอียด **(ห้ามใช้ any)**
- กำหนด Types ให้กับ Zustand Store State และ Actions ชัดเจน

<u>การส่งงาน</u>

1) ไฟล์เว็บไซต์ตามกำหนดที่เสร็จสมบูรณ์ พร้อมไฟล์ประกอบทั้งหมด (ยกเว้น node_modules)

2) ไฟล์ตัวอย่างข้อมูล .json หรือ รูปแบบอื่นเป็นไฟล์ (หากพัฒนา Backend สำหรับ API สามารถ แนบ folder ชุดของ API ได้)

3) ไฟล์ทั้งหมดในข้อ 1) และ 2) ให้ใส่ Folder โดยตั้งชื่อเป็น **mini_proj2_รหัสนักศึกษา.zip** (zip ไฟล์งานทั้งหมด)

<u>สิ่งที่ต้องส่ง</u>

- ZIP files ทั้งหมด (ไฟล์ **mini_proj2_รหัสนักศึกษา.zip) (ลบโฟลเดอร์ node_modules และ dist ออกก่อน Zip)**
- ภาพถ่ายหน้าจอ (Screenshots): ภาพบันทึกหน้าจอระบบงานจริงครบทุกหน้า (วางไว้ในโฟลเดอร์ screenshots/ ภายในไฟล์ Zip)
- Repository Link: แนบลิงก์ GitHub / GitLab Repository ในช่องส่งงาน พร้อมสิทธิ์การเข้าถึง (Public หรือเชิญผู้สอน) (ถ้ามี)
- **Live Deployment (ตัวเลือกแนะนำ)**: แนบลิงก์ที่ Deploy บน Vercel, Netlify หรือ Cloudflare Pages เพื่อใช้ประกอบการตรวจรอบพรีเซนต์

ตัวอย่าง **Public APIs:**

- **E-commerce:** `FakeStoreAPI` (fakestoreapi.com)
- สูตรอาหาร: `Spoonacular API` (spoonacular.com/food-api), `TheMealDB`
- เกม: `RAWG Video Games Database API` (rawg.io/apidocs)
- หนังสือ: `Open Library APIs` (openlibrary.org/developers/api), `Google Books API`
- ข่าว: `NewsAPI` (newsapi.org)

เกณฑ์การประเมินผล (**Rubric: 10 คะแนน**)

มิติการประเมิน	เกณฑ์การพิจารณา	คะแนน
1. Modern Decoupled State Architecture	แยกหน้าที่ระหว่าง Zustand (Client State + Persist) และ TanStack Query (Server State + Cache + Stale Time) ได้ถูกต้อง ไม่ซ้ำซ้อน	**3**
2. Routing & Navigation	ใช้ React Router v7 จัดทำ Root Layout, <Outlet/>, <NavLink>, และ Dynamic Route (:id) ได้สมบูรณ์ไร้ข้อผิดพลาด	**2**
3. UI/UX, Loading & Error States	มี Skeleton/Spinner ระหว่างโหลดข้อมูล, มี Error State ชัดเจน, Responsive Design สวยงามด้วย Tailwind CSS / DaisyUI	**2**
4. Feature Completeness (5 Pages)	ครบทั้ง 5 หน้าตามข้อกำหนด (Items, Detail, Collection, About, Extended Page) มีข้อมูลไม่ต่ำกว่า 20 รายการ	**2**
5. Code Quality & TypeScript	โครงสร้างโฟลเดอร์เป็นระเบียบ สะอาด มีการกำหนด Type/Interface ครบถ้วน ไม่พบการใช้ any	**1**
รวมคะแนนเต็ม		**10**